

Token & Positional Embeddings

in the Transformer

A ground-up study note

L2 norms, the $\sqrt{d_{\text{model}}}$ scaling, sinusoidal vs. learned encodings

Built from a working session that started with one suspicious line of code and unwound every assumption behind it: what an L2 norm is, why a random vector's length lands near $\sqrt{d}$, why $\sin^2 \theta + \cos^2 \theta = 1$, how positional encodings store position, and whether the token embedding and the positional encoding are magnitude-compatible.

Revisit this whenever the concepts need rebuilding from scratch.

Contents

1	The starting point: two modules and one worry	2
2	What an L2 norm is	2
3	Why a random vector's norm lands near $\sqrt{d}$	2
3.1	The hidden identity: $\mathbb{E}[X^2] = \text{Var}(X) + (\mathbb{E}[X])^2$	2
3.2	Applying it to an embedding row	3
3.3	Why $\sqrt{d}$ specifically: norm follows dimensionality	3
4	The $\sqrt{d_{\text{model}}}$ scaling of the token embedding	4
4.1	The bug this avoids	4
4.2	“Scale down then scale up” — is it a pointless round trip?	4
5	Sinusoidal positional encoding from the ground up	5
5.1	What it is	5
5.2	The load-bearing identity: $\sin^2 \theta + \cos^2 \theta = 1$	5
5.3	The norm: $\ \text{PE}\ = \sqrt{d_{\text{model}}/2}$	6
5.4	Same length, different direction: how position is stored	6
6	Learned positional encoding	7
6.1	L2 norm: same recipe, different numbers	7
6.2	What the <code>forward</code> method does, step by step	8
7	Are the two modules magnitude-compatible?	8
8	One-page summary	9

1 The starting point: two modules and one worry

A transformer turns a sequence of token IDs into vectors in two steps that are then *added* together:

$$\text{input to layer 1} = \underbrace{\text{token embedding}}_{\text{which word}} + \underbrace{\text{positional encoding}}_{\text{where in the sequence}}.$$

Because the two are added, a single principle governs everything in this note: **their magnitudes must be comparable**, or the larger one drowns out the other. “Magnitude” here means the $L2$ norm of each vector. The whole session was an attempt to make that one sentence precise.

The two modules under study:

- **TokenEmbedding** — a learned lookup table, scaled by $\sqrt{d_{\text{model}}}$.
- **LearnedPositionalEncoding** — a learned lookup table indexed by position.

The thread of the whole note. Token embedding norm $\approx \sqrt{d_{\text{model}}}$, positional encoding norm $\approx \sqrt{d_{\text{model}}}$ (learned) or exactly $\sqrt{d_{\text{model}}/2}$ (sinusoidal). Same order of magnitude $\Rightarrow$ neither signal erases the other when they are summed.

2 What an L2 norm is

The $L2$ norm (Euclidean norm), written $\|v\|$ or $\|v\|_2$, is the **length** of a vector — the straight-line distance from the origin to the point it represents. For $v = (v_1, v_2, \dots, v_d)$:

$$\|v\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_d^2}.$$

The “2” is the $p = 2$ case of the general L_p norm $(\sum_i |v_i|^p)^{1/p}$. It is just the Pythagorean theorem generalized from two dimensions to d : square each coordinate, add, take the square root.

3 Why a random vector’s norm lands near $\sqrt{d}$

This single fact is used everywhere below, so it is worth deriving carefully.

3.1 The hidden identity: $\mathbb{E}[X^2] = \text{Var}(X) + (\mathbb{E}[X])^2$

The step people skip is that the expected value of the square is *not* the square of the expected value. The exact gap between them is the variance. Starting from the definition of variance and expanding:

$$\begin{aligned} \text{Var}(X) &= \mathbb{E}[(X - \mu)^2] & \mu &= \mathbb{E}[X] \\ &= \mathbb{E}[X^2 - 2\mu X + \mu^2] & &\text{expand} \\ &= \mathbb{E}[X^2] - 2\mu \mathbb{E}[X] + \mu^2 & &\text{linearity} \\ &= \mathbb{E}[X^2] - 2\mu^2 + \mu^2 = \mathbb{E}[X^2] - \mu^2. \end{aligned}$$

Rearranging gives the master identity:

$$\boxed{\mathbb{E}[X^2] = \text{Var}(X) + \mu^2}$$

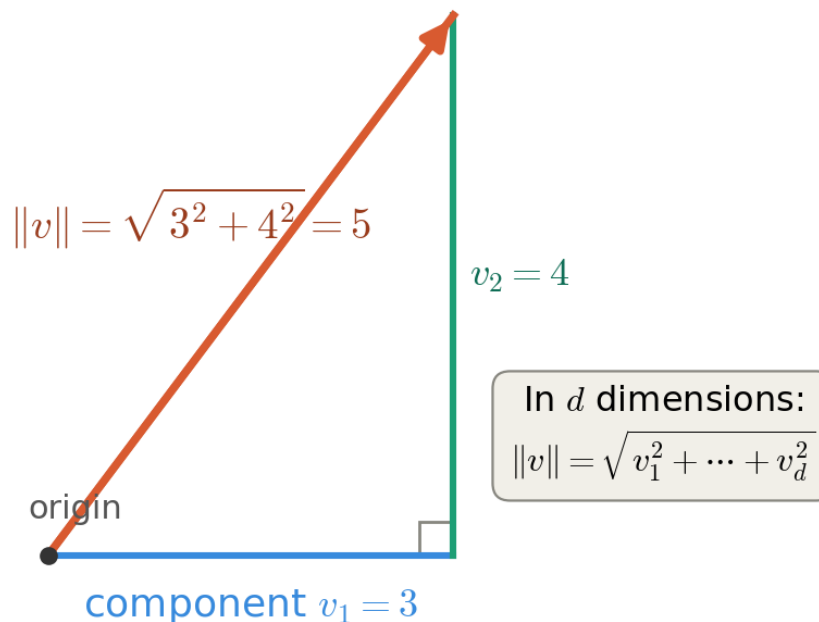


Figure 1: The L2 norm is the length of the vector. In two dimensions it is the hypotenuse of a right triangle with legs equal to the components; in d dimensions it is the same recipe with more terms under the root.

3.2 Applying it to an embedding row

PyTorch’s default `nn.Embedding` init draws each entry from $\mathcal{N}(0, 1)$, so $\mu = 0$ and $\text{Var} = 1$, hence $\mathbb{E}[v_i^2] = 1 + 0^2 = 1$. By linearity of expectation (no independence needed),

$$\mathbb{E}[\|v\|^2] = \mathbb{E}[\sum_i v_i^2] = \sum_i \mathbb{E}[v_i^2] = \sum_i 1 = d, \quad \text{so} \quad \|v\| \approx \sqrt{d}.$$

If instead each entry is drawn from $\mathcal{N}(0, 1/d)$, then $\mathbb{E}[v_i^2] = 1/d$, so $\mathbb{E}[\|v\|^2] = d \cdot \frac{1}{d} = 1$ and $\|v\| \approx 1$.

One-line intuition. $\mathbb{E}[X^2]$ is the average squared distance from *zero*; $\text{Var}(X)$ is the average squared distance from the *mean*. When the mean is zero, they coincide — which is why a zero-mean variable has $\mathbb{E}[X^2] = \text{Var}(X)$.

3.3 Why $\sqrt{d}$ specifically: norm follows dimensionality

The deeper reason the answer is $\sqrt{d}$ rather than any other number: **the norm of a vector grows as $\sqrt{(\text{number of dimensions})}$ whenever each dimension contributes a roughly fixed amount of “energy.”** It is the Pythagorean theorem at scale — doubling the dimensions multiplies the norm by only $\sqrt{2} \approx 1.41$, not by 2.

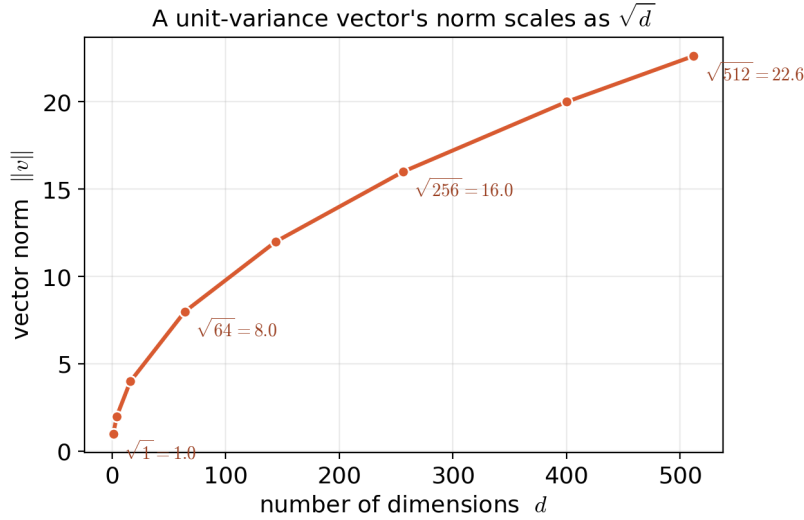


Figure 2: Norm versus dimension count for unit-variance components. The sub-linear $\sqrt{d}$ growth is why the scaling factor used later is $\sqrt{d_{\text{model}}}$, not d_{model} .

4 The $\sqrt{d_{\text{model}}}$ scaling of the token embedding

The original transformer (*Attention Is All You Need*, §3.4) multiplies the token embeddings by $\sqrt{d_{\text{model}}}$. A faithful implementation pairs this with an initialization at variance $1/d_{\text{model}}$:

1. Initialize weights with standard deviation $1/\sqrt{d_{\text{model}}}$, i.e. $\mathcal{N}(0, 1/d_{\text{model}}) \Rightarrow$ row norm ≈ 1 .
2. Multiply the output by $\sqrt{d_{\text{model}}}$ $\Rightarrow$ row norm $\approx \sqrt{d_{\text{model}}}$.

4.1 The bug this avoids

With PyTorch’s *default* $\mathcal{N}(0, 1)$ init, the rows already have norm $\approx \sqrt{d_{\text{model}}}$. Multiplying by $\sqrt{d_{\text{model}}}$ then overshoots to norm $\approx d_{\text{model}}$ — a factor of $\sqrt{d_{\text{model}}}$ too large. At $d_{\text{model}} = 512$ that is a norm of 512 against a sinusoidal PE’s $\sqrt{256} = 16$: the token signal buries the positional one by a factor of 32.

Documentation caveat. A stale inline comment in the code claimed the overshoot drowns the PE “ $\sim 12\times$.” The true factor is $\sqrt{d_{\text{model}}}$, which is ≈ 22.6 at $d_{\text{model}} = 512$ (the “12” only fits $d_{\text{model}} = 144$, since $\sqrt{144} = 12$). The factor varies with the model dimension and should be written $\sim \sqrt{d_{\text{model}}}\times$.

4.2 “Scale down then scale up” — is it a pointless round trip?

A natural objection: the init scales the rows *down* to norm 1, and the forward pass scales them back *up* to $\sqrt{d_{\text{model}}}$. For this module’s output alone, the two exactly cancel. The reason it is not redundant is **weight tying**: the same stored matrix is reused as the output projection that produces logits,

$$\text{logits} = (\text{final hidden}) \cdot W_{\text{emb}}^{\top},$$

and the output path does *not* multiply by $\sqrt{d_{\text{model}}}$.

- As the **output projection**, the matrix should sit at the small scale (norm ≈ 1); otherwise the logit dot-products are $\sqrt{d_{\text{model}}}$ times too large and saturate the softmax at initialization.
- As the **input embedding**, it should sit at norm $\approx \sqrt{d_{\text{model}}}$ to match the positional encoding.

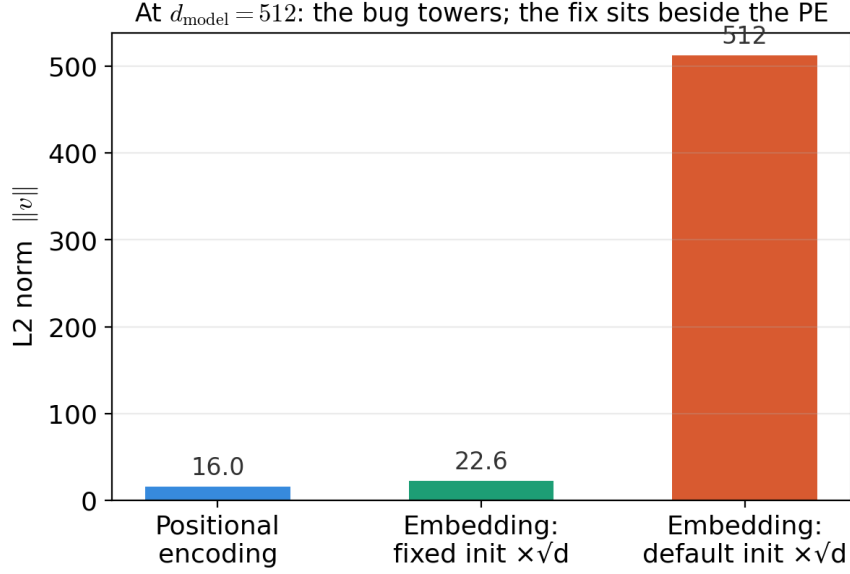


Figure 3: Three regimes at $d_{\text{model}} = 512$. The bug (default init $\times \sqrt{d_{\text{model}}}$) reaches norm 512 and dwarfs the positional encoding; the fix (variance-1/ d_{model} init $\times \sqrt{d_{\text{model}}}$) lands at ≈ 22.6 , right beside the PE.

The resolution: store the matrix at the small scale (good for the output side) and apply the $\sqrt{d_{\text{model}}}$ correction *only on the input path*. The down-scale is the resting state of the shared parameter; the up-scale is a local fix-up the output path skips. A second, subtler non-cancellation: the multiply also rescales the *gradients* reaching the embedding, changing its effective learning rate — so even untied, “the output is identical” is not quite the whole story once training begins.

Verdict. The recipe is meaningful exactly when the embedding weight is tied to the output head. If the weights are *not* tied, the down-then-up scaling is a genuine no-op for this module, and you should either tie them or drop the recipe.

5 Sinusoidal positional encoding from the ground up

5.1 What it is

Attention is order-agnostic — to it, “the cat sat” and “sat cat the” are the same bag of words. So we inject position by adding a position-dependent vector to each token. For position pos , the sinusoidal encoding builds a d_{model} -long vector whose entries alternate sine and cosine, each pair at its own frequency:

$$\text{PE}(pos, 2i) = \sin(\omega_i pos), \quad \text{PE}(pos, 2i + 1) = \cos(\omega_i pos), \quad \omega_i = \frac{1}{10000^{2i/d_{\text{model}}}}.$$

Dimensions $2i$ and $2i + 1$ share the *same* frequency ω_i — one sine, one cosine. They are a matched pair. Early pairs wiggle fast; later pairs wiggle slowly.

5.2 The load-bearing identity: $\sin^2 \theta + \cos^2 \theta = 1$

A frequent confusion: it is the *squared* sum that equals 1, not $\sin \theta + \cos \theta$. Numerically, at $\theta = 0.6$: $\sin \theta + \cos \theta = 0.56 + 0.83 = 1.39$ (not 1), whereas $\sin^2 \theta + \cos^2 \theta = 0.32 + 0.68 = 1.00$.

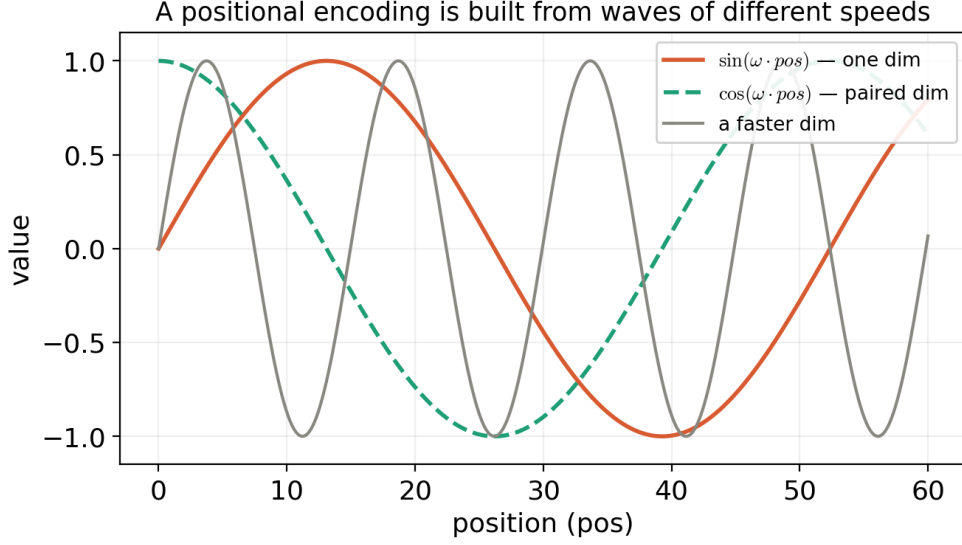


Figure 4: Each PE entry is a sine or cosine of the position. A sine and its paired cosine share a frequency (a quarter-cycle apart); other dimensions use faster or slower frequencies.

Why is the squared version always 1? It is the Pythagorean theorem in disguise. With $\sin \theta = o/h$ and $\cos \theta = a/h$ in a right triangle,

$$\sin^2 \theta + \cos^2 \theta = \frac{o^2}{h^2} + \frac{a^2}{h^2} = \frac{o^2 + a^2}{h^2} = \frac{h^2}{h^2} = 1,$$

since $o^2 + a^2 = h^2$ by Pythagoras. Equivalently: the point $(\cos \theta, \sin \theta)$ lies on the circle of radius 1, so its squared distance from the origin is 1.

5.3 The norm: $\|\text{PE}\| = \sqrt{d_{\text{model}}/2}$

Stringing it together: the L2 norm needs the sum of the squares of all entries. Group them into the $d_{\text{model}}/2$ sine–cosine pairs. Each pair contributes exactly 1. Therefore

$$\|\text{PE}\|^2 = \sum_{i=0}^{d_{\text{model}}/2-1} (\sin^2(\omega_i \text{ pos}) + \cos^2(\omega_i \text{ pos})) = \underbrace{1 + 1 + \dots + 1}_{d_{\text{model}}/2 \text{ ones}} = \frac{d_{\text{model}}}{2}, \quad \|\text{PE}\| = \sqrt{\frac{d_{\text{model}}}{2}}.$$

This is *exact*, and identical for every position. (An equivalent “mean-square $\frac{1}{2}$ ” argument: each individual entry’s square averages to $\frac{1}{2}$ via $\sin^2 \theta = \frac{1 - \cos 2\theta}{2}$, so d_{model} entries give $d_{\text{model}}/2$. The pairing argument is cleaner because it needs no averaging.)

5.4 Same length, different direction: how position is stored

Because every positional vector has the *same* length $\sqrt{d_{\text{model}}/2}$, they all lie on one sphere (a circle in 2-D, a hypersphere in d_{model} -D). What differs between positions is only the *direction* the vector points. The smallest case $d_{\text{model}} = 2$ makes this literal: $\text{PE} = [\sin(\omega \text{ pos}), \cos(\omega \text{ pos})]$ is a point that rides around the unit circle as pos advances — the radius is locked, the angle is the position.

Using many frequencies (like clock hands at different speeds) avoids the wrap-around of a single circle: fast hands distinguish nearby positions, slow hands keep far-apart positions distinct, and the combination of all hand-angles pins down each position uniquely — while every hand keeps a fixed length, so the total norm stays fixed.

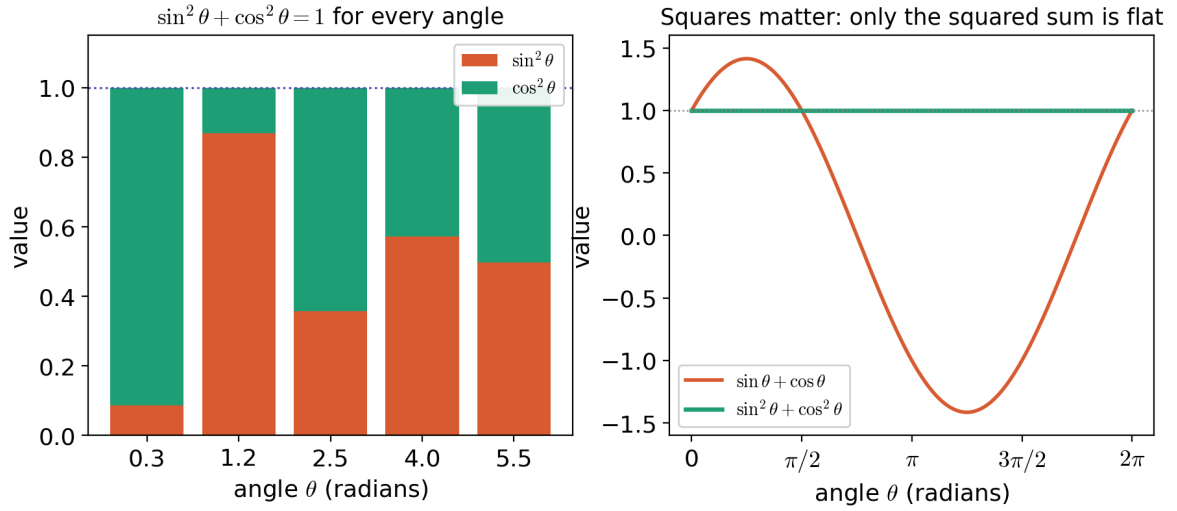


Figure 5: Left: for any angle, $\sin^2 \theta$ and $\cos^2 \theta$ stack to exactly 1. Right: $\sin \theta + \cos \theta$ (no squares) wanders between $\pm\sqrt{2}$ and only crosses 1 at isolated points; only the squared sum is flat at 1.

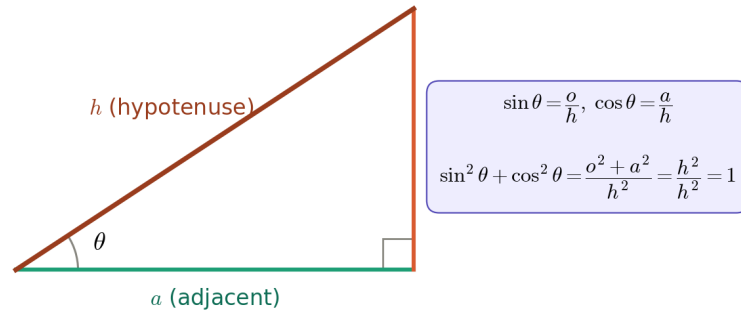


Figure 6: The identity is the Pythagorean theorem divided through by h^2 .

6 Learned positional encoding

The learned variant replaces the fixed $\sin / \cos$ formula with a trainable lookup table: `nn.Embedding(max_seq_len, d_model)`. Row i is the (learned) positional vector for position i .

6.1 L2 norm: same recipe, different numbers

The norm is still $\sqrt{\text{sum of squares of the row}}$ — but the numbers are learned parameters, so there is no closed-form value. Two regimes:

- **At initialization**, PyTorch fills the table from $\mathcal{N}(0, 1)$, so by the $\mathbb{E}[\|v\|^2] = d$ result, each row has norm $\approx \sqrt{d_{\text{model}}}$. Unlike sinusoidal, this is not exact — each row is a different random draw, so the norms scatter. The squared norm follows a χ^2 distribution with d_{model} degrees of freedom (mean d_{model}), and the norm itself concentrates around $\sqrt{d_{\text{model}}}$ with a standard deviation of only ≈ 0.7 , almost independent of d_{model} .
- **After training**, backprop is free to lengthen or shorten each row independently; the tidy bell smears out and different positions can take different lengths. No identity holds anything fixed.

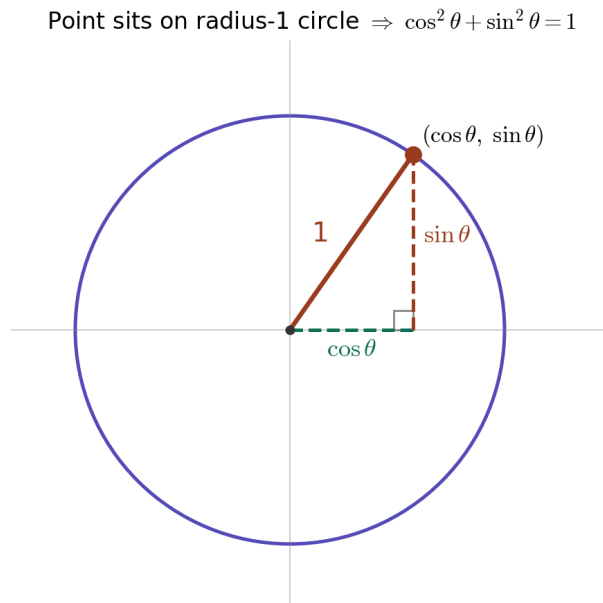


Figure 7: The unit-circle view: $\cos \theta$ and $\sin \theta$ are the coordinates of a point on a circle of radius 1; the radius is the fixed hypotenuse, so $\cos^2 \theta + \sin^2 \theta = 1$ always.

6.2 What the forward method does, step by step

For an input $\mathbf{x}$ of shape (batch, seq_len):

1. `seq_len = x.size(1)` — read only the sequence length; the token ID *values* are ignored (position cares *where*, not *what*).
2. `positions = torch.arange(seq_len, device=x.device)` — build the index list $[0, 1, \dots, \text{seq_len}-1]$ on the same device as $\mathbf{x}$.
3. `self.position_embedding(positions)` — look up those rows, giving shape (seq_len, d_{model}).
4. `.unsqueeze(0)` — add a leading dimension to get (1, seq_len, d_{model}) so the *same* positional vectors broadcast across every sentence in the batch.

Edge case. The table has exactly `max_seq_len` rows. If an input arrives with `seq_len > max_seq_len`, `arange` produces indices past the end of the table and the lookup throws an index error — this scheme only handles sequences up to the length it was sized for. Sinusoidal encoding, by contrast, is defined for any position.

7 Are the two modules magnitude-compatible?

Putting the corrected `TokenEmbedding` next to the `LearnedPositionalEncoding`:

- **Token embedding:** init std $1/\sqrt{d_{\text{model}}}$ $\Rightarrow$ row norm 1; forward $\times \sqrt{d_{\text{model}}}$ $\Rightarrow$ output norm $\approx \sqrt{d_{\text{model}}}$.
- **Learned PE:** default $\mathcal{N}(0, 1)$ init $\Rightarrow$ row norm $\approx \sqrt{d_{\text{model}}}$; returned raw (no scaling).

Both land at $\approx \sqrt{d_{\text{model}}}$. At $d_{\text{model}} = 512$ that is ≈ 22.6 each — a 1:1 match, even tighter than the token-plus-sinusoidal pairing the docstring was written for ($\sqrt{d_{\text{model}}}$ vs. $\sqrt{d_{\text{model}}}/2$, a $\sqrt{2}$ gap).

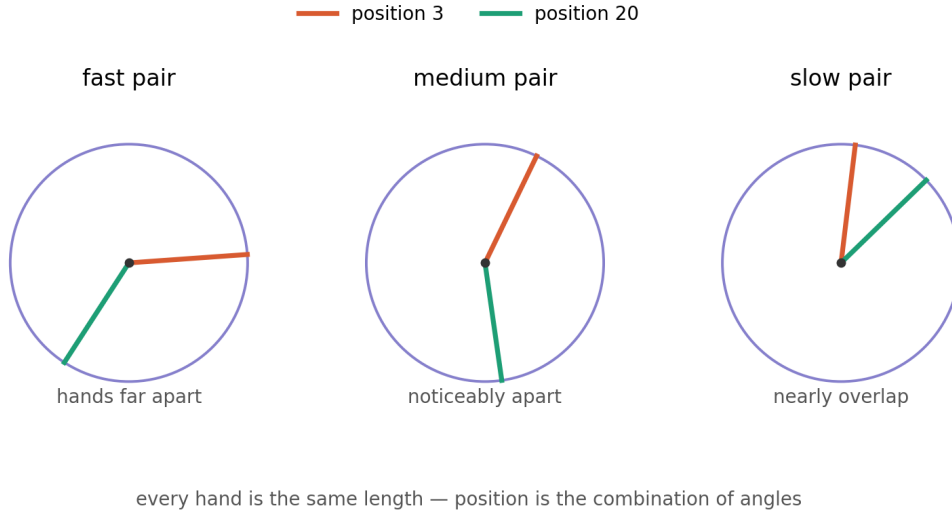


Figure 8: Two positions on three frequency dials. The fast hand swings far between positions 3 and 20; the slow hand barely moves. Position is the combination of angles; all hands keep equal length, so the total magnitude is unchanged.

Answer: yes, at initialization. Three caveats: (1) it is an initialization-time, in-expectation statement — the learned PE drifts freely during training (which is fine; the model self-corrects); (2) both norms have $\approx \pm 0.7$ scatter; (3) the token-embedding doc-string still references *sinusoidal* $\sqrt{d_{\text{model}}}/2$ and the stale “12×”, which describe a different partner than the learned PE actually wired up here. None of these are code bugs — they are documentation fixes plus the reminder that the match is a starting condition, not a guarantee.

8 One-page summary

The logical chain, end to end.

1. L2 norm = $\sqrt{\text{sum of squares}}$ — the length of a vector.
2. A vector of d unit-variance entries has $\mathbb{E}[\|v\|^2] = d$, so $\|v\| \approx \sqrt{d}$ (from $\mathbb{E}[X^2] = \text{Var} + \mu^2$).
3. Token embedding: init at variance $1/d_{\text{model}}$ (norm 1), then $\times \sqrt{d_{\text{model}}} \Rightarrow$ norm $\sqrt{d_{\text{model}}}$. The down-then-up is non-trivial only under weight tying.
4. Sinusoidal PE: entries are sin / cos pairs; each pair’s squares sum to 1 (Pythagoras), and $d_{\text{model}}/2$ pairs give norm exactly $\sqrt{d_{\text{model}}}/2$, identical for every position; position is encoded in *direction*, not length.
5. Learned PE: same norm recipe on learned parameters; $\approx \sqrt{d_{\text{model}}}$ at init (tight χ bell), free after training; capped at `max_seq_len`.
6. Compatibility: corrected token embedding ($\sqrt{d_{\text{model}}}$) and learned PE ($\sqrt{d_{\text{model}}}$) are matched 1:1 at init — neither drowns the other.

Reference: Vaswani et al., Attention Is All You Need (2017), §3.4 for the $\sqrt{d_{\text{model}}}$ scaling and the sinusoidal positional encoding formula.

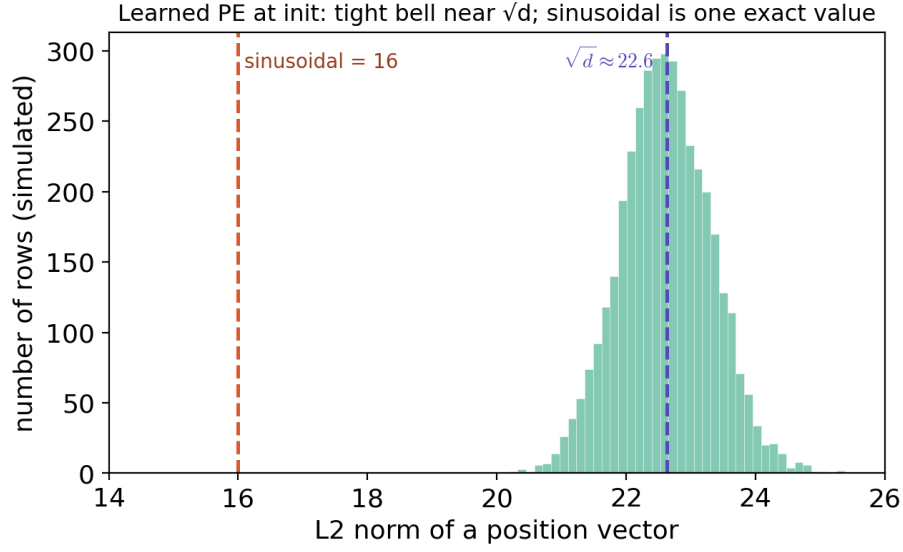


Figure 9: Simulated L2 norms of 4000 random 512-dimensional rows. They form a narrow bell around $\sqrt{512} \approx 22.6$ — a fuzzy shell, not the razor-thin single value $\sqrt{d_{\text{model}}/2} = 16$ that sinusoidal encoding would give.

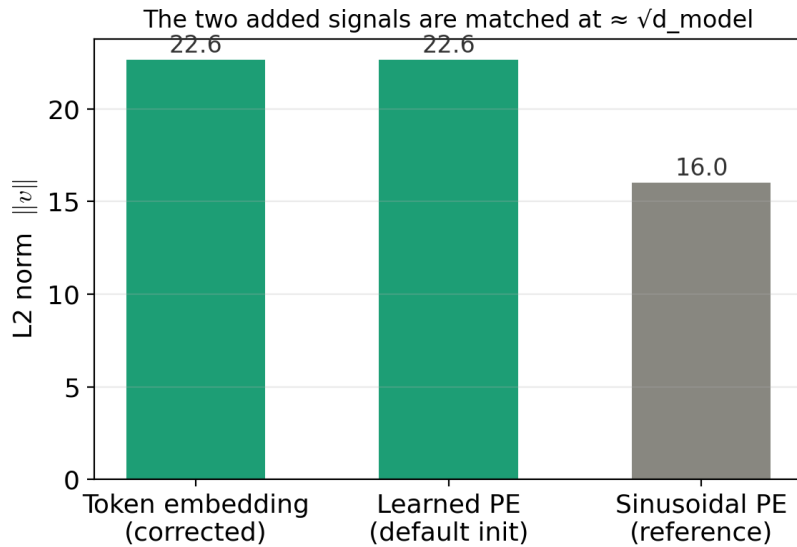


Figure 10: At $d_{\text{model}} = 512$ the token embedding and the learned positional encoding sit at the same norm ≈ 22.6 . The sinusoidal value 16 is shown for reference.